

The application layer

Because of the simplicity of the *SearchService*, we can directly move on to the application layer without worrying about any domain services, beyond domain objects.

The application layer consists of usual DTOs and controllers. *UserRecord* is a DTO that just acts as a data container without any logic. One useful feature is that it translates the object into JSON format.

```
class UserRecord{
  constructor(user){
    this.name = user.getName().getValue();
    this.phoneNumber = user.getPhoneNumber().
getValue();
    this.since = user.getSince();
  }
  toJSON(){
    return {
      name: this.name,
      phoneNumber: this.phoneNumber,
      since: this.since
    }
  }
}

export default UserRecord;
```

Users is another DTO that acts as a container for multiple *UserRecord* objects.

```
class Users{
  constructor(records){
    this.records = records;
    this.size = records.length;
  }
  toJSON(){
    return {
      size: this.size,
      users: this.records
    }
  }
}

export default Users;
```

Now comes the controller. It acts as a gateway between the domain layer and the user interface layer.

Begin with importing the necessary domain classes.

```
import repo from '../data/UserRepository.js'
import Users from './Users.js'
import UserRecord from './UserRecord.js';
```

Create the *UserController* class with an externally visible function *searchByPhoneNumber* that accepts input, interacts with the domain layer for request processing and returns DTOs.

```
class UserController{
  async searchByPhoneNumber(phoneNumber){
    let entities = await repo.
findByPhoneNumber(phoneNumber);
    let dtos = [];
    for(let entity of entities){
      dtos.push(new UserRecord(entity).toJSON())
    }
    return new Users(dtos);
  }
}

const controller = new UserController();
export default controller;
```

Note that the *UserController* is also a singleton, though not a strict requirement.

Since we are exposing the *SearchService* as a REST endpoint to the user interfaces, let's plug that in. It is here that ExpressJS helps us. The first step is to import the *express* object.

```
import express from 'express';
```

Import the *body-parser* and *cors* to parse the JSON bodies and handle CORS, respectively.

```
import bodyParser from 'body-parser';
import cors from 'cors';
```

Import the controller that we already developed.

```
import controller from './app/UserController.js'
```

Once the imports are ready, create and configure the *express* object in the usual way.

```
const app = express();
const port = process.env.PORT || 8080;
app.use(cors());
app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());
```

Note that we made the PORT a configurable variable. It's always a good design practice to externalize anything that is possible to make the system as flexible as possible.